

Lab Procedure

Thermocouple

Introduction

Ensure the following:

1. You have reviewed the **Application Guide – Thermocouple**
2. Make sure the **Mechatronic Sensors Trainer** is out of its case; it is powered using the provided USB cable connected to the PC.
 - a. The LED next to the USB C port will stay white after the touch screen starts loading.
 - b. The load screen with the Quanser Logo should disappear after a few seconds and you should see some evenly spaced crosses on a black background on the touch screen.
3. Using the provided thermocouple's yellow connector, look for the side with the + and – signs. Connect the thermocouple to the **Sensors Trainer** by matching the – in the connector to the – on the **Thermocouple connector** on the screen side of the device.

Exploring Thermocouple Inputs

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **File > Open Folder...**, and browse to the directory containing the python files and lab procedure for the Thermocouple lab (`.../sensors_trainer/1_fundamentals/thermocouple/hardware/python`). Open this directory.
2. Open **thermocouple.py**. The device will get initialized in the line:
with SensorsTrainer() as sensors:
3. With the thermocouple connected to the device, run **thermocouple.py** using the  button or the terminal. This will open the **Temperature** plot that shows both the temperature of the thermocouple chip, located inside the case, as well as the temperature of the probe itself.
4. Zoom into the plot if needed, you can use the scroll wheel on your mouse, and you can drag around the plot to see the area you are interested in. Look at the plot while warming the probe with your hands and observe how the temperature changes.
5. Click back on the terminal in Visual Studio Code and stop the script using **Ctrl+C** and clicking enter when prompted (otherwise the script stops after 150 seconds automatically).
6. Let the temperature probe get back to room temperature.

7. Change the **simulationTime** at the top of the file to 10 seconds, and run the code again, warming the thermocouple with your hands as you go. Once the data stops updating, the code stops. Keep the probe display window open. Take a screenshot if needed as it might close.
8. Observe the probe signal, zoom in if needed, what is the minimum temperature change that is read? Click back on your terminal and click enter to close the window.
9. Disconnect the Sensors Trainer from the PC. We want the device to cool down slightly. Also disconnect the thermocouple from the device.
10. Open the Thermocouple Chip's Datasheet located in the thermocouple folder (.../sensors_trainer/1_fundamentals/thermocouple). Read page 1, 8 and 9 (until before the Conversion Functions section) which have a quick summary of the chip that reads the temperature from the thermocouple. Note that the version we are using is the one for K type thermocouples.
11. Does the information on the datasheet match what you observed for the minimum temperature change?
12. In the concept review, you reviewed the different types of junctions for thermocouples. If you have a voltmeter, set it to check for continuity, which indicates whether there is an electrical path connecting the two voltmeter leads. First check between the two leads in the thermocouple's yellow connector, what happens and why do you think that is?
13. Check for continuity between one of the leads in the connector and the metal casing of the probe. What happens? Based on that, what type of thermocouple junction type does the thermocouple have?

Testing the Thermocouple

14. Update the **simulationTime** to 60 seconds.
15. The datasheet mentions the following: "While the temperature at the cold end fluctuates, the device continues to accurately sense the temperature difference at the opposite end." We want to test if that is the case.
16. Reconnect the thermocouple to the device and plug in the device again to turn it on.
17. Place the thermocouple probe on a flat surface where it will not be disturbed.
18. Run **thermocouple.py** using the  button or the terminal, if the values start in the thousands and then go back to environment temperatures, stop the code using **Ctrl+C** and clicking enter when prompted and start it again.
19. Let the code run for the full minute. Do not close the probe window. Note that when you start zooming out of the scope, a little **A**  appears at the bottom left of the window, click it to size the data to the screen. Take a screenshot of your plot to show results.

20. Observe both the chip temperature as well as the probe temperature. What do you observe based on the statement from the datasheet, do you find it to be accurate?
21. Get a cup of water that is warmer than room temperature. You could have both a cold and hot water and use the cold water to cool down the thermocouple too.
22. Change the **simulationTime** to 60 seconds.
23. Run **thermocouple.py** using the  button or the terminal. Let it read room temperature for a few seconds and then immerse the thermocouple probe in the liquid with a temperature difference. Observe how it reaches a final temperature.
24. Let the code finish running. Do not close the scope window. Note that when you start zooming out of the scope, a little **A** () appears at the bottom left of the window, click it to size the data to the screen. Make your scope window full screen and take a screenshot of your plot to show results.
25. Using the screenshot of your plot, find how long it takes to reach the final stable temperature value. Calculate the time constant, which is how long it takes to reach 63.2% of the final temperature.
26. Run the code again and explore different ways of placing the probe into the water to see if this changes the temperature readings. , For example, test it if only the top of the probe is immersed, or only a side of it. What do you observe?
27. Stop, save and close your program.
28. Power OFF the Mechatronic Sensors Trainer by disconnecting its USB C cable, disconnect any external sensors and pack them along the base and the USB cables in the provided case.